ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



Quản Lý File
Quản Lý Tiến Trình

Báo Cáo Đồ Án Số 1

Hệ Điều Hành - CSC10007

Sinh viên thực hiện:

24120010 - Trần Văn Tấn Khôi

24120035 - Lê Thành Đạt

24120361 - Trịnh Tuấn Kiệt

Giảng viên hướng dẫn:

Phạm Tuấn Sơn

Lê Viết Long

Thành phố Hồ Chí Minh, Ngày 7 tháng 11 năm 2026

BẢNG PHÂN CHIA CÔNG VIỆC

MSSV	Họ và tên	Phân công công việc	%
24120010	Trần Văn Tấn Khôi	• Khảo sát `ofile[]`, `ftable.file[]` và `struct file`. • Phân tích luồng `sys_open` và so sánh `open`/`dup`/`fork`. • Viết chương 2 - Quản lý file.	100%
24120035	Lê Thành Đạt	• Khảo sát `pid`, `state`, `context`, `trapframe` và `fork`. • Phân tích cơ chế định thời `scheduler`, `sched` và `yield`. • Viết chương 3 - Quản lý tiến trình.	100%
24120361	Trịnh Tuấn Kiệt	• Thiết kế ABI và cài đặt system call `sysinfo` trong kernel. • Cài đặt snapshot, đồng bộ lock và đếm `nsched`. • Viết `user/sysinfotest.c`, chạy kiểm thử QEMU và viết chương 4.	100%

MỤC LỤC

MỤC LỤC ii

1 Mục tiêu và phạm vi 1

2 Quản lý file 3

2.1 Hai tầng quản lý file: `ofile[]` và `ftable.file[]` . . . 4

2.1.1 File descriptor là chỉ số, không phải file . . . 4

2.1.2 `ftable` và `structfile` . . . 4

2.1.3 Vòng đời của `ref` . . . 5

2.2 Luồng `syscall open` . . . 6

2.2.1 Từ user space vào kernel . . . 6

2.2.2 Các bước trong `sys_open` . . . 6

2.2.3 Xử lý lỗi . . . 8

2.3 So sánh file descriptor trong `open`, `dup` và `fork` . . . 8

2.3.1 Bảng so sánh . . . 9

2.3.2 Hệ quả thực tế . . . 9

2.3.3 Vì sao `dup` gọi `fdalloc` trước `filedup` . . . 10

2.3.4 Trường hợp pipe . . . 10

3 Quản lý tiến trình 11

3.1 Các trường `pid`, `state`, `context` và `struct proc` . . . 12

3.1.1 `pid` và `state` . . . 12

3.1.2 `context` và `trapframe`: hai loại ngữ cảnh khác nhau . . . 12

3.1.3 Đồng bộ hóa . . . 14

3.2 Luồng `syscall fork` . . . 14
3.2.1 Các bước thực hiện . . . 14
3.2.2 Vì sao `fork` "trả về hai lần" . . . 15
3.2.3 Xử lý lỗi . . . 15
3.3 Cơ chế định thời: `scheduler()`, `sched()`, `yield()` . . . 15
3.3.1 `scheduler()` . . . 15
3.3.2 `sched()` . . . 16
3.3.3 `yield()` và preemption . . . 16
3.3.4 `sleep`, `wakeup`, `exit`, `wait` . . . 17
3.3.5 Nhận xét . . . 17

4 Cài đặt và kiểm thử `sysinfo` 18
4.0.1 Thiết kế dữ liệu và ABI . . . 19
4.0.2 Bộ đệm snapshot và đồng bộ . . . 20
4.0.3 Thu thập dữ liệu . . . 21
4.0.4 Đếm số lần định thời . . . 22
4.0.5 `user/sysinfotest.c` và kết quả . . . 22

5 Kết Luận 26

DANH MỤC CÁC BẢNG 29

DANH MỤC CÁC HÌNH ẢNH 30

CHƯƠNG 1

Mục tiêu và phạm vi

Mục 4.1 yêu cầu tìm hiểu cách xv6 quản lý file và tiến trình, cụ thể là: hai bảng lưu thông tin file (`ofile[]` của `struct proc` và `file[]` của `ftable`), luồng xử lý của hai system call `open` và `fork`, cách dùng file descriptor ở ba con đường `open`/`dup`/`fork`, các trường pid, `state`, `context` của tiến trình, và cơ chế định thời (`scheduler`, `sched`, `yield`).

Mục 4.2 yêu cầu thêm system call `sysinfo` trả về số open file toàn hệ thống kèm thông tin (type, mode, reference count, danh sách pid đang tham chiếu) và danh sách tiến trình không `UNUSED` (pid, state, parent pid, số lần được định thời, danh sách fd), cùng chương trình `user/sysinfotest.c` kiểm thử càng nhiều tình huống càng tốt. Nhóm cài đặt theo hướng snapshot best-effort, không giữ đồng thời `ftable.lock` và `p->lock`, và kiểm thử bằng cách so sánh delta giữa các lần gọi thay vì số tuyệt đối.

Yêu cầu	Nội dung	Mục tương ứng
4.1	`ofile[]` của `struct proc`, `file[]` của `ftable`	2.1
4.1	Luồng `syscall open`	2.2
4.1	So sánh file descriptor trong `open`, `dup`, `fork`	2.3
4.1	`pid`, `state`, `context` của `struct proc`	2.4
4.1	Luồng `syscall fork`	2.5
4.1	Cơ chế `scheduler()`, `sched()`, `yield()`	2.6
4.2	Số open file và thông tin từng file	3.1–3.3
4.2	Danh sách tiến trình và thông tin từng tiến trình	3.1–3.3
4.2	`user/sysinfotest.c` và kết quả kiểm thử	3.5

Bảng 1.1: Phân công và đối chiếu mục nội dung

CHƯƠNG 2

Quản lý file

2.1. Hai tầng quản lý file: `ofile[]` và `ftable.file[]`

2.1.1. File descriptor là chỉ số, không phải file

Mỗi tiến trình sở hữu một mảng `ofile[]` nằm ngay trong `struct proc` (`kernel/proc.h:107`), với kích thước `NOFILE = 16` (`kernel/param.h:3`). Phần tử `ofile[fd]` là một con trỏ tới `struct file`, hoặc bằng 0 nếu descriptor đó chưa được dùng. Như vậy **file descriptor chỉ là một số nguyên nhỏ, đóng vai trò chỉ số** vào bảng riêng của tiến trình; bản thân nó không chứa thông tin gì về file.

Điều này giải thích vì sao mọi system call nhận file descriptor (`read`, `write`, `close`, `dup`, `fstat`...) đều phải đi qua bước kiểm tra `argfd()` (`kernel/sysfile.c:22`): hàm này kiểm tra `fd` nằm trong khoảng `[0, NOFILE)` và `ofile[fd]` khác 0, rồi mới trả về con trỏ `struct file` tương ứng. Một `fd` âm, quá lớn, hoặc đã bị `close` đều bị từ chối ngay tại đây.

2.1.2. `ftable` và `structfile`

Ở tầng thứ hai, toàn hệ thống có đúng một bảng `ftable` (`kernel/file.c:18-21`). Điểm cần lưu ý về mặt cú pháp: `ftable` là **một biến toàn cục** có kiểu struct không tên, chứ không phải một kiểu tên là `struct ftable`. Đề bài viết "`file[]` của `struct ftable`" là cách gọi theo nghĩa chức năng; trong mã nguồn không tồn tại `struct ftable`.

Mã nguồn 1: `struct ftable`

```
struct {
    struct spinlock lock;
    struct file file[NFILE];
} ftable;
```

`NFILE = 100` (`kernel/param.h:4`), tức tối đa 100 open file trên toàn hệ thống, trong khi mỗi tiến trình chỉ được mở tối đa 16. Hai giới hạn này độc lập: một tiến trình có thể chạm `NOFILE` trước, hoặc cả hệ thống chạm `NFILE` trước, tùy tình huống.

`struct file` (`kernel/file.h:1-10`) là **open file description** - đối tượng trung tâm của mọi thao tác đọc/ghi:

Trường	Ý nghĩa
`type`	Một trong `FD_NONE`, `FD_PIPE`, `FD_INODE`, `FD_DEVICE`
`ref`	Số tham chiếu đang trỏ tới open file này
`readable`	Có được đọc hay không, suy ra từ `omode` lúc `open`
`writable`	Có được ghi hay không, suy ra từ `omode` lúc `open`
`pipe`	Con trỏ tới `struct pipe` khi `type == FD_PIPE`
`ip`	Con trỏ tới inode khi `type == FD_INODE` hoặc `FD_DEVICE`
`off`	Offset hiện tại, chỉ dùng cho `FD_INODE`
`major`	Số hiệu thiết bị chính, chỉ dùng cho `FD_DEVICE`

Bảng 2.1: Các trường trong cấu trúc quản lý file

Đề bài nhắc tới "mode" của file. Trong xv6 **không có trường `mode` nguyên vẹn** được lưu lại; thay vào đó `sys_open` chuyển `omode` thành hai cờ boolean `readable` và `writable`, và hai cờ này được kiểm tra lại trong `fileread()`/`filewrite()` trước khi thực hiện thao tác. Cách biểu diễn này đủ cho xv6 vì kernel không cần phân biệt `O_RDONLY` với `O_RDWR` sau khi đã mở.

Một điểm tinh tế: `ref` đếm **số tham chiếu từ các file descriptor**, không phải số tiến trình. Nếu một tiến trình gọi `dup()` ba lần trên cùng một fd, `ref` tăng thêm ba dù chỉ có một tiến trình. Hệ quả này rất quan trọng khi cài đặt `sysinfo` ở mục 4.2: khi liệt kê "các tiến trình đang tham chiếu tới open file", một tiến trình có thể xuất hiện một lần nhưng đóng góp nhiều đơn vị vào `ref`.

2.1.3. Vòng đời của `ref`

Ba hàm trong `kernel/file.c` quản lý `ref`, tất cả đều thao tác dưới `ftable.lock`:

- `filealloc()` (`file.c:56`) quét tuần tự `ftable.file[]`, chọn phần tử đầu tiên có `ref == 0`, đặt `ref = 1` rồi trả về. Nếu hết slot, trả 0. Hàm **không** khởi tạo các trường còn lại; `sys_open` chịu trách nhiệm gán `type`, `ip`, `off`, `readable`, `writable` sau đó.
- `filedup(f)` (`file.c:74`) tăng `ref` thêm 1, `panic` nếu `ref < 1`. Đây là thao tác "chia sẻ"

một open file đã tồn tại.

- `fileclose(f)` (`file.c:86`) giảm `ref`. Nếu vẫn còn lớn hơn 0 thì chỉ trả về. Khi `ref` về 0, hàm **sao toàn bộ `struct file` ra biến cục bộ `ff`**, đặt `type = FD_NONE`, rồi **thả `ftable.lock` trước khi** gọi `pipeclose()` hoặc `iput()`. Việc thả lock sớm là cố ý: `iput()` có thể phải chờ đĩa và đi ngủ, không được giữ spinlock trong lúc đó.

Như vậy một slot trong `ftable` được coi là tự do khi `ref == 0` (và `type == FD_NONE`). Đây cũng là tiêu chí để đếm "số open file toàn hệ thống" ở mục 4.2: đếm các phần tử có `ref > 0` và `type != FD_NONE`, không đếm toàn bộ 100 slot.

2.2. Luồng `syscall open`

2.2.1. Từ user space vào kernel

Một lời gọi `open()` không đi thẳng vào kernel: prototype nằm trong `user/user.h`, còn phần thân là stub trong `user/usys.S` (sinh tự động bởi `user/usys.pl`). Stub nạp số hiệu system call vào `a7`, tham số vào `a0`–`a5`, rồi thực hiện `ecall`.

Sau trap, `usertrap()` gọi `syscall()` (`kernel/syscall.c:162`). Hàm này đọc số hiệu từ `p->trapframe->a7`, tra bảng `syscalls[num]`, gọi hàm xử lý, rồi ghi kết quả vào `p->trapframe->a0`. Theo ABI RISC-V, `a0` là thanh ghi trả về, và `usertrapret()` khôi phục nó cho chương trình người dùng.

2.2.2. Các bước trong `sys_open`

`sys_open()` nằm ở `kernel/sysfile.c:305`. Luồng chính gồm các bước sau:

Bước	Việc làm	Ghi chú
1	`argint(1, &omode)`, `argstr(0, path, MAXPATH)`	Lấy `omode` và copy path từ user space vào buffer kernel
2	`begin_op()`	Mở một transaction của lớp log; mọi thao tác file system phải nằm trong cặp `begin_op/end_op`
3	Nếu có `O_CREATE`: `create(path, T_FILE, 0, 0)`	Trả về inode **đã được khoá**
4	Ngược lại: `namei(path)` rồi `ilock(ip)`	`namei` chỉ tìm, chưa khoá; phải khoá để đọc metadata an toàn
5	Nếu là `T_DIR` và `omode != O_RDONLY`: từ chối	Không cho ghi trực tiếp vào thư mục
6	Nếu là `T_DEVICE`: kiểm tra `major` trong `[0, NDEV)`	Tránh gọi driver không tồn tại
7	`filealloc()` rồi `fdalloc(f)`	Cấp open file description, sau đó cấp slot trong `ofile[]`
8	Gán `type`, `ip`, `off = 0`, `readable`, `writable`	Với `FD_DEVICE` thì lưu `major` và không dùng `off`
9	Nếu có `O_TRUNC` và là `T_FILE`: `itrunc(ip)`	Cắt file về 0 byte
10	`iunlock(ip)`, `end_op()`, trả `fd`	Kết thúc transaction và trả descriptor

Bảng 2.2: Các bước trong luồng `syscall open`

Hai chi tiết đáng chú ý:

- `create()` trả về inode đã khoá, còn `namei()` thì không. Vì vậy nhánh `O_CREATE` **không** gọi `ilock()` lại, còn nhánh còn lại thì có. Đây là lý do hai nhánh trông không đối xứng

trong code.

- `fdalloc()` (`kernel/sysfile.c:40`) quét từ `fd = 0` và chọn slot trống đầu tiên. Nhờ vậy ba descriptor `0`, `1`, `2` trở thành quy ước chuẩn cho stdin/stdout/stderr, và một fd vừa `close` sẽ được cấp lại cho lần `open` kế tiếp.

2.2.3. Xử lý lỗi

`sys_open` có nhiều nhánh thoát sớm, và mỗi nhánh phải hoàn trả tài nguyên đúng thứ tự:

- `argstr`/`argint` thất bại → trả `-1` ngay, chưa chiếm tài nguyên.
- `create`/`namei` thất bại → `end_op()` rồi trả `-1`.
- Ghi vào thư mục hoặc `major` không hợp lệ → `iunlockput(ip)` (mở khoá và giảm `ref` inode) rồi `end_op()`.
- `filealloc`/`fdalloc` thất bại → `fileclose(f)` nếu đã có `struct file`, rồi `iunlockput(ip)` và `end_op()`.

Trường hợp `fdalloc` thất bại sau khi `filealloc` thành công là an toàn: lúc này `type` vẫn là `FD_NONE` và chưa gán `ip`, nên `fileclose` chỉ đưa `ref` về 0 mà không gọi `iput`/`pipeclose`.

2.3. So sánh file descriptor trong `open`, `dup` và `fork`

2.3.1. Bảng so sánh

Tiêu chí	`open()`	`dup(fd)`	`fork()`
Cấp `struct file` mới?	Có, qua `filealloc`	Không, dùng lại của `fd`	Không, dùng lại của cha
Cấp fd mới?	Có, fd trống nhỏ nhất	Có, fd trống nhỏ nhất	Không cấp mới; con có **cùng số fd**
Ảnh hưởng `ref`	Đặt `ref = 1`	`ref` tăng 1	`ref` tăng 1 cho **mỗi** fd được kế thừa
Offset `off`	Riêng (với `FD_INODE`, khởi tạo 0)	**Dùng chung** với `fd` gốc	**Dùng chung** với cha
Phạm vi ảnh hưởng	Chỉ tiến trình gọi	Chỉ tiến trình gọi	Cha và con
Vị trí code	`sysfile.c:305`	`sysfile.c:55`	`proc.c:282`

Bảng 2.3: So sánh file descriptor trong `open`, `dup`, `fork`

Cả ba con đường đều dẫn tới `ofile[fd]` trỏ vào một `struct file`, nhưng khác nhau ở chỗ **open file description được tạo mới hay được chia sẻ**. `open` tạo mới; `dup` và `fork` chia sẻ.

2.3.2. Hệ quả thực tế

Vì `dup` và `fork` chia sẻ `off`, hai descriptor được tạo theo hai cách này sẽ **nối tiếp nhau** khi đọc/ghi:

- `dup` rồi đọc lần lượt từ hai fd sẽ thấy dữ liệu nối tiếp, không đọc lại từ đầu. - Cha và con sau `fork` cùng đọc một file sẽ chia nhau các byte, đúng như hành vi của pipeline trong shell. - Ngược lại, `open` cùng một path hai lần tạo ra **hai open file description độc lập**, mỗi cái có `off` riêng, dù cả hai cùng trỏ tới một inode trong bộ nhớ.

`close` cũng cần được hiểu theo mô hình này. `sys_close` (`sysfile.c:98`) chỉ đặt `ofile[fd] = 0` rồi gọi `fileclose(f)`. Nếu `ref` vẫn còn (do `dup`, `fork`, hoặc tiến trình khác đang giữ), open file description **không bị huỷ** và các descriptor còn lại tiếp tục hoạt động bình thường. Chỉ khi `ref`

về 0 thì tài nguyên mới được giải phóng (`iput` với inode, `pipeclose` với pipe).

2.3.3. Vì sao `dup` gọi `fdalloc` trước `filedup`

Trong `sys_dup` (`sysfile.c:55`), thứ tự là `argfd` → `fdalloc` → `filedup`. Nếu `fdalloc` thất bại (bảng `ofile` đã đầy), hàm trả `-1` mà **chưa** tăng `ref`, nên không có tham chiếu bị rò. Nếu làm ngược lại, mỗi lần `dup` thất bại sẽ để lại một `struct file` bị giữ vĩnh viễn - một dạng rò rỉ tài nguyên.

Với `fork`, thứ tự ngược lại về mặt logic: con đã có bản sao `ofile[]`, nên với mỗi slot khác 0, `filedup` được gọi để tăng `ref` tương ứng. Nếu `uvmcopy` thất bại trước đó, `freeproc` sẽ dọn sạch con và không có `ref` nào bị tăng.

2.3.4. Trường hợp pipe

Với `FD_PIPE`, `struct file` dùng `f->pipe` thay cho `ip`/`off`. `sys_pipe` (`sysfile.c:478`) tạo **hai** open file description - đầu đọc và đầu ghi - cùng trỏ tới một `struct pipe`; `fork` chia sẻ cả hai, nên cha con giao tiếp được qua pipe. Đầu đọc chỉ nhận EOF khi mọi bản sao của đầu ghi đã đóng, vì vậy `ref` và các cờ `readopen`/`writeopen` trong `struct pipe` đều phải được theo dõi.

CHƯƠNG 3

Quản lý tiến trình

3.1. Các trường pid, state, context và struct proc

3.1.1. pid và state

`pid` là định danh duy nhất của tiến trình, được cấp trong `allocproc()` (`proc.c:110`) và được dùng bởi `getpid`, `kill`, `wait`, cũng như để cha nhận diện con sau `fork`. `struct proc` còn lưu `parent` – con trỏ tới tiến trình cha – nhưng trường này được bảo vệ bởi `wait_lock` chứ không phải `p->lock`, vì `wait()` quét toàn bảng để tìm con.

`state` có kiểu `enum procstate` với sáu giá trị (`proc.h:82`). Bảng dưới đây tóm tắt ai đặt trạng thái nào:

Trạng thái	Ý nghĩa	Nơi đặt
`UNUSED`	Slot chưa được dùng	`freeproc()` khi `wait()` dọn con
`USED`	Đã cấp phát nhưng chưa sẵn sàng chạy	`allocproc()`
`RUNNABLE`	Sẵn sàng, đang chờ CPU	`fork()`, `userinit()`, `yield()`, `wakeup()`, `kill()` (nếu nạn nhân đang ngủ)
`RUNNING`	Đang chạy trên một CPU	`scheduler()` khi chọn được tiến trình
`SLEEPING`	Đang ngủ trên một kênh chờ	`sleep()`
`ZOMBIE`	Đã thoát, chờ cha `wait()`	`exit()`

Bảng 3.1: Các trạng thái của tiến trình trong `struct proc`

Tiến trình `ZOMBIE` vẫn chiếm slot và giữ `pid` nhưng không chạy được; chỉ khi cha gọi `wait()` thì `freeproc()` mới đưa nó về `UNUSED`. Vì vậy tiến trình không được `wait` sẽ thành zombie.

3.1.2. context và trapframe: hai loại ngữ cảnh khác nhau

Đây là điểm dễ nhầm nhất trong `struct proc`. xv6 lưu **hai** ngữ cảnh riêng biệt:

Tiêu chí	`struct context`	`struct trapframe`
Mục đích	Chuyển ngữ cảnh **kernel** → **kernel**	Lưu trạng thái khi **user** → **kernel** và ngược lại
Nội dung	`ra`, `sp`, `s0`–`s11` (14 thanh ghi)	Toàn bộ thanh ghi user, `epc`, `kernel_satp`, `kernel_sp`, `kernel_trap`, `kernel_hartid`
Ai lưu/khôi phục	`swtch()` trong `kernel/swtch.S`	`uservec`/`userret` trong `kernel/trampoline.S`
Vị trí	Nằm trong `struct proc` (và trong `struct cpu`)	Nằm ở một trang riêng, được ánh xạ vào page table của tiến trình
Khi nào dùng	Mỗi lần scheduler đổi tiến trình	Mỗi lần vào/ra kernel

Bảng 3.2: So sánh giữa `struct context` và `struct trapframe`

Vì sao `context` chỉ cần 14 thanh ghi? Vì `swtch()` là một lời gọi hàm C bình thường. Theo quy ước gọi hàm của RISC-V, các thanh ghi **caller-saved** (`a0`–`a7`, `t0`–`t6`) đã được trình biên dịch tự động lưu vào stack của kernel thread trước khi gọi `swtch()`. Chỉ các thanh ghi **callee-saved** (`s0`–`s11`), `ra` và `sp` là cần được lưu tường minh, vì chúng phải giữ nguyên giá trị qua lời gọi. Nhờ vậy `swtch()` chỉ cần vài chục lệnh load/store, rất rẻ so với việc lưu toàn bộ thanh ghi.

`trapframe` thì khác: nó chứa **toàn bộ** trạng thái của chương trình người dùng tại thời điểm trap, kể cả `epc` (địa chỉ lệnh bị ngắt) và các thanh ghi `a0`–`a7`. Khi system call trả về, `syscall()` đã ghi kết quả vào `p->trapframe->a0`; `usertrapret()` và `userret` khôi phục toàn bộ thanh ghi từ trapframe rồi `sret` về user space.

`struct cpu` (`proc.h:22`) cũng có một `struct context` riêng. Đó là ngữ cảnh của **kernel thread điều phối** trên CPU đó: khi `swtch(&c->context, &p->context)` được gọi, ngữ cảnh của scheduler được lưu vào `c->context`, còn ngữ cảnh của tiến trình được nạp từ `p->context`. Nhờ vậy mỗi CPU có thể quay lại đúng vòng lặp `scheduler()` sau khi tiến trình nhường CPU.

3.1.3. Đồng bộ hóa

`struct proc` có một spinlock riêng `p->lock`. Các trường `state`, `pid`, `chan`, `killed`, `xstate` phải được đọc/ghi khi giữ lock này. Riêng `parent` được bảo vệ bởi `wait_lock` toàn cục, vì `wait()` cần quét nhất quán quan hệ cha–con trong khi `fork()` và `exit()` có thể thay đổi nó.

Các trường còn lại như `ofile[]`, `cwd`, `sz`, `pagetable`, `trapframe`, `name` là **riêng của tiến trình**: chỉ chính tiến trình đó sửa, nên không cần `p->lock`. Đây là lý do `fork()` có thể đọc `p->ofile[i]` mà không cần giữ lock của cha. Tuy nhiên, khi một tiến trình khác (ví dụ system call `sysinfo` ở mục 4.2) muốn đọc `ofile[]` của tiến trình khác, nó phải cẩn thận với thứ tự lock để tránh deadlock.

3.2. Luồng `syscall fork`

3.2.1. Các bước thực hiện

`sys_fork()` (`kernel/sysproc.c:26`) chỉ đơn giản gọi `fork()` trong `kernel/proc.c:282`. Luồng chính:

1. `allocproc()` (`proc.c:110`): quét bảng `proc` tìm slot `UNUSED`, đặt `state = USED`, cấp `pid`, cấp một trang `trapframe`, tạo page table rỗng, và khởi tạo `context` với `ra = forkret`, `sp = kstack + PGSIZE`. Hàm trả về với `p->lock` **đang được giữ**.
2. `uvmcopy()`: copy toàn bộ vùng nhớ người dùng của cha sang con. xv6 bản này **copy eager**, không dùng copy-on-write.
3. **Copy trapframe**: `*(np->trapframe) = *(p->trapframe)`, sau đó đặt `np->trapframe->a0 = 0`. Đây là chi tiết khiến `fork()` trả về 0 ở tiến trình con.
4. **Kế thừa file**: với mỗi `i` mà `p->ofile[i] != 0`, gán `np->ofile[i] = filedup(p->ofile[i])`. Nhờ đó con có cùng số fd và cùng open file description với cha.
5. **Kế thừa thư mục hiện tại**: `np->cwd = idup(p->cwd)`.
6. **Gán cha**: `acquire(&wait_lock); np->parent = p; release(&wait_lock)`. Việc dùng `wait_lock` thay vì `p->lock` giúp `wait()` quét bảng mà không phải xen kẽ hai loại lock.
7. **Đặt** `RUNNABLE` dưới `np->lock`, rồi trả `pid` cho cha.

3.2.2. Vì sao `fork` "trả về hai lần"

Trong cùng một lời gọi `fork()`, có hai luồng thực thi tiếp tục từ hai kernel stack khác nhau:

- **Cha** tiếp tục ngay trong hàm `fork()`, nhận `pid` của con và trả về user space qua đường `usertrapret()` thông thường.
- **Con** chưa từng chạy trong `fork()`. Khi scheduler chọn nó, `swtch()` nạp `p->context` có `ra = forkret`, nên luồng thực thi bắt đầu tại `forkret()` (`proc.c:530`). Hàm này thả `np->lock` (lần đầu tiên còn gọi `fsinit`), rồi gọi `usertrapret()` để quay về user space. Vì `trapframe` của con là bản copy của cha với `a0 = 0`, lệnh `ecall` của con "trả về" giá trị 0.

Nói cách khác, `fork()` không thực sự chạy hai lần; nó chỉ tạo ra hai ngữ cảnh cùng quay về một điểm trong chương trình người dùng, với hai giá trị `a0` khác nhau.

3.2.3. Xử lý lỗi

Nếu `uvmcopy` thất bại (hết bộ nhớ), `fork()` gọi `freeproc(np)` để trả slot về `UNUSED`, thả `np->lock` và trả `-1`. Lúc này `np->ofile[]` vẫn toàn 0 vì vòng `filedup` chưa chạy, nên không có `ref` nào bị tăng. Nếu `allocproc()` thất bại ngay từ đầu, hàm trả `-1` mà không cần dọn gì.

Một chi tiết nhỏ nhưng quan trọng: `fork()` copy `trace_mask` từ cha sang con (`proc.c:305`). Đây là trường được thêm ở Lab 0 để hỗ trợ system call `trace`; nếu thiếu bước này, tiến trình con sẽ mất trạng thái trace của cha.

3.3. Cơ chế định thời: `scheduler()`, `sched()`, `yield()`

3.3.1. `scheduler()`

Mỗi CPU sau khi khởi tạo đều gọi `scheduler()` (`proc.c:450`) và không bao giờ quay lại. Vòng lặp của nó:

1. Bật interrupt (`intr_on()`) để tránh deadlock khi mọi tiến trình đang ngủ.
2. Quét tuần tự toàn bộ `proc[0..NPROC-1]`, với mỗi slot thì `acquire(&p->lock)`.
3. Nếu `p->state == RUNNABLE`, đặt `p->state = RUNNING`, gán `c->proc = p`, rồi `swtch(&c->context, &p->context)`.

4. Khi tiến trình nhường CPU và quay lại `scheduler()`, `c->proc` được đặt lại 0 và `p->lock` được thả.
5. Nếu không tìm được tiến trình nào, CPU thực hiện `wfi` (wait for interrupt) để tiết kiệm năng lượng.

Việc giữ `p->lock` trong suốt quá trình chọn và chuyển ngữ cảnh bảo đảm **một tiến trình không thể được chọn bởi hai CPU cùng lúc**. Đây là lý do xv6 chạy đúng trên máy nhiều nhân dù không có hàng đợi chạy (run queue) phức tạp: toàn bộ bảng `proc` đóng vai trò hàng đợi, và lock của từng slot là điểm đồng bộ.

Cơ chế này cũng có chi phí: mỗi lần chọn tiến trình là một lần quét `NPROC = 64` slot, và không có độ ưu tiên. Đổi lại, code rất ngắn và dễ kiểm chứng.

3.3.2. `sched()`

`sched()` (`proc.c:497`) là cầu nối giữa kernel thread của tiến trình và `scheduler()`. Trước khi `swtch()`, nó kiểm tra bốn điều kiện bất biến và `panic` nếu vi phạm:

- Đang giữ `p->lock`.
- `mycpu()->noff == 1`, tức chỉ còn đúng một spinlock đang giữ (chính là `p->lock`).
- `p->state != RUNNING`, vì tiến trình đã phải đổi trạng thái trước khi nhường CPU.
- Interrupt đang tắt, để tránh bị ngắt giữa lúc đang chuyển ngữ cảnh.

Sau đó `sched()` lưu `intena` (trạng thái interrupt trước đó của kernel thread), gọi `swtch(&p->context, &mycpu()->context)`, rồi khôi phục `intena` khi được scheduler chọn lại. Việc lưu/khôi phục này cần thiết vì `intena` thuộc về kernel thread chứ không thuộc về CPU.

Bốn điều kiện trên biến `sched()` thành một "cửa ải" an toàn: mọi đường đi vào scheduler (`yield`, `sleep`, `exit`) đều phải chuẩn bị đúng trạng thái và lock trước khi gọi.

3.3.3. `yield()` và preemption

`yield()` (`proc.c:518`) thực hiện đúng ba việc: `acquire(&p->lock)`, `p->state = RUNNABLE`, `sched()`, rồi `release(&p->lock)` sau khi được chọn lại.

`yield()` được gọi từ trap handler khi có **timer interrupt**: `trap.c:81` trong `usertrap()` và

`trap.c:155` trong `kerneltrap()`. Nhờ vậy xv6 có định thời **preemptive round-robin**: sau mỗi chu kỳ timer, tiến trình đang chạy bị đưa về `RUNNABLE` và nhường CPU cho tiến trình kế tiếp trong bảng. Nếu không có timer interrupt, một vòng lặp vô hạn trong user space sẽ chiếm CPU mãi mãi.

Đây cũng là điểm tự nhiên để đếm "số lần được định thời" cho mục 4.2: mỗi khi `scheduler()` đặt `state = RUNNING`, đó là một sự kiện lập lịch. Tuy nhiên cần thống nhất định nghĩa (đếm lúc được chọn chạy, hay đếm lúc nhường CPU) vì hai cách lệch nhau đúng một đơn vị ở thời điểm quan sát.

3.3.4. `sleep`, `wakeup`, `exit`, `wait`

Bốn hàm này hoàn thiện vòng đời trạng thái:

- `sleep(chan, lk)` (`proc.c:554`): giữ `p->lock`, thả `lk`, gán `p->chan = chan`, đặt `SLEEPING`, gọi `sched()`. Khi được đánh thức, nó xoá `chan`, thả `p->lock` và lấy lại `lk`. Thứ tự này bảo đảm không mất tín hiệu `wakeup` giữa lúc kiểm tra điều kiện và lúc ngủ.
- `wakeup(chan)` (`proc.c:585`): quét bảng, với mỗi tiến trình `SLEEPING` có `p->chan == chan` thì đặt `RUNNABLE`.
- `exit(status)` (`proc.c:352`): đóng toàn bộ `ofile[]`, `iput(cwd)`, chuyển con cho `init`, đánh thức cha, đặt `ZOMBIE`, rồi `sched()` và không bao giờ quay lại.
- `wait(addr)` (`proc.c:396`): quét con, nếu có con `ZOMBIE` thì copy `xstate` ra user space, gọi `freeproc()` (đưa về `UNUSED`) và trả `pid`. Nếu chưa có con nào thoát, nó `sleep(p, &wait_lock)`.

3.3.5. Nhận xét

Cơ chế định thời của xv6 tối giản nhưng đầy đủ: không độ ưu tiên, không run queue nhiều mức, mỗi CPU tự quét bảng nên tải có thể lệch giữa các nhân. Tính đúng đắn vẫn được bảo đảm nhờ `p->lock` và các bất biến của `sched()`, và toàn bộ cơ chế chỉ dựa trên một trường `state` cùng một spinlock cho mỗi tiến trình.

CHƯƠNG 4

Cài đặt và kiểm thử `sysinfo`

4.0.1. Thiết kế dữ liệu và ABI

`struct sysinfo` được khai báo một lần trong `kernel/sysinfo.h` (`kernel/sysinfo.h:20-45`) và được dùng chung cho cả kernel lẫn chương trình người dùng. Cấu trúc chỉ gồm các mảng tĩnh, không chứa con trỏ, nên có thể copy nguyên khối bằng `copyout()` mà không phụ thuộc địa chỉ kernel.

Thành phần	Trường	Ý nghĩa
2*`struct sysinfo`	`nfile`, `files[100]`	Số open file và thông tin từng open file
	`nproc`, `procs[64]`	Số tiến trình không `UNUSED` và thông tin từng tiến trình
4*`struct sysinfo_file`	`type`	`FD_PIPE`, `FD_INODE` hoặc `FD_DEVICE`
	`readable`, `writable`	Hai cờ thay cho `mode`; phản ánh quyền đọc/ghi lúc `open`
	`ref`	Số descriptor đang trỏ tới open file này
	`npid`, `pids[64]`	Số pid hợp lệ và danh sách pid đang tham chiếu, không trùng lặp
4*`struct sysinfo_proc`	`pid`, `state`	Định danh và trạng thái tại thời điểm chụp
	`parent_pid`	Pid của cha; bằng 0 nếu không có cha (ví dụ `init`)
	`nsched`	Số lần `scheduler()` đã chọn tiến trình này
	`nfd`, `fds[16]`	Số fd hợp lệ và danh sách chỉ số fd đang mở

Bảng 4.1: Cấu trúc dữ liệu cho hệ thống thống kê tiến trình và file

Các giới hạn được đặt bằng hằng số của kernel: `SYSINFO_MAX_FILES = NFILE = 100`, `SYSINFO_MAX_PROCS`

= SYSINFO_MAX_PIDS = NPROC = 64, SYSINFO_MAX_FDS = NOFILE = 16. Kích thước thực tế là $4 + 100 \times 276 + 4 + 64 \times 84 = 32.984$ byte, khớp với dòng sizeof(struct sysinfo) = 32984 bytes khi chạy thử. state được lưu dưới dạng int thay vì enum procstate để ABI không phụ thuộc vào kiểu enum của từng lần biên dịch; giá trị số vẫn giữ nguyên thứ tự của enum.

Về phía người dùng, user/user.h khai báo trước struct sysinfo và prototype int sysinfo(struct sysinfo *) (user/user.h:2,27), còn user/usys.pl sinh stub entry("sysinfo") (user/usys.pl:40). Trong kernel, số hiệu là SYS_sysinfo = 23 (kernel/syscall.h:24), được thêm vào bảng dispatch, bảng tên dùng cho trace, và prototype (kernel/syscall.c:105,132,158). Chương trình _sysinfotest được thêm vào UPROGS (Makefile:199).

4.0.2. Bộ đệm snapshot và đồng bộ

struct sysinfo có kích thước 32.984 byte, lớn hơn nhiều so với kernel stack 4 KB. Nếu khai báo nó như một biến cục bộ trong sys_sysinfo() thì stack sẽ tràn ngay khi system call được gọi. Vì vậy bộ đệm snapshot được đặt ở BSS (kernel/sysproc.c:112) và được bảo vệ bởi một spinlock riêng snapshot_lock (kernel/sysproc.c:113). sysinfoinit() khởi tạo lock này trong main() sau fileinit() (kernel/main.c:30, kernel/sysproc.c:117-120).

Luồng của sys_sysinfo() (kernel/sysproc.c:179-193):

1. Lấy địa chỉ user bằng argaddr(0, &addr).
2. acquire(&snapshot_lock) để bảo đảm chỉ một lời gọi dùng bộ đệm chung tại một thời điểm.
3. Gọi collect(&snapshot) để điền dữ liệu.
4. copyout() toàn bộ cấu trúc sang user space.
5. Thả lock và trả 0, hoặc -1 nếu copyout() thất bại.

Việc giữ snapshot_lock trong lúc copyout() 32 KB là một đánh đổi có chủ ý. Cách này đơn giản, không cần cấp phát động và không thể thất bại vì hết trang; đổi lại, các lời gọi sysinfo() bị tuần tự hoá và CPU phải tắt interrupt trong suốt thời gian copy. Với bài lab, chi phí này chấp nhận được; hướng cải thiện được nêu ở mục 4.6.

Một chi tiết liên quan đến BSS của chương trình người dùng: user/sysinfotest.c cũng đặt các snapshot (si, prev) ở BSS vì user stack chỉ có một trang. Trong cấu hình LAB_SYSCALL, uvmalloc()

bỏ qua `memset(mem, 0, sz)` (`kernel/vm.c:271-273`), nên BSS của user **không được đảm bảo bằng 0**. Vì vậy test tự gán `failures = 0` và luôn gọi `getinfo()` trước khi đọc snapshot, không dựa vào khởi tạo tĩnh.

4.0.3. Thu thập dữ liệu

Việc thu thập chia làm hai pha, mỗi pha chỉ giữ một loại lock.

Pha 1 - bảng open file. `filesnapshot()` (`kernel/file.c:33-52`) giữ `ftable.lock`, duyệt `ftable.file[0..NFILE)`, bỏ qua các slot có `ref == 0` hoặc `type == FD_NONE` (slot đang được `filealloc()` cấp nhưng chưa gán loại), rồi copy `type`, `readable`, `writable`, `ref`, đặt `npid = 0` và lưu địa chỉ `struct file *` vào mảng `fileptr[]`. Hàm trả về `nfile` rồi thả `ftable.lock`. Việc lưu con trỏ chỉ nhằm mục đích so khớp ở pha 2; con trỏ này không được gửi ra user space.

Pha 2 - bảng tiến trình. `collect()` (`kernel/sysproc.c:140-174`) duyệt `proc[0..NPROC)`. Với mỗi slot, hàm giữ `p->lock`, bỏ qua tiến trình `UNUSED`, copy `pid`, `state`, `parent_pid`, `nsched`, rồi duyệt `ofile[0..NOFILE)`: fd nào khác 0 được ghi vào `fds[]`, sau đó so khớp con trỏ `p->ofile[i]` với `fileptr[]` để thêm pid vào danh sách của open file tương ứng. `addpid()` (`kernel/sysproc.c:122-133`) bỏ qua pid đã có, nên một tiến trình chỉ xuất hiện một lần trong `pids[]` dù nó `dup()` cùng một file nhiều lần.

Thứ tự lock. `sys_sysinfo()` giữ `snapshot_lock` ở ngoài cùng; bên trong, pha 1 lấy `ftable.lock`, pha 2 lấy `p->lock`, nhưng hai lock này **không bao giờ được giữ đồng thời**. Lý do là `fork()` đang giữ `p->lock` của tiến trình con rồi gọi `filedup()` để lấy `ftable.lock` (`kernel/proc.c:312-313`), tạo ra thứ tự `p->lock → ftable.lock`. Nếu `sysinfo` giữ `ftable.lock` rồi chờ `p->lock` thì sẽ tạo chu trình và deadlock. Vì `snapshot_lock` chỉ được lấy trước hai lock kia và không có đường nào lấy nó sau `ftable.lock`/`p->lock`, đồ thị lock vẫn là một DAG.

Hệ quả về tính nhất quán. Đây là snapshot **best-effort**, không phải ảnh chụp nguyên tử của toàn hệ thống. Giữa pha 1 và pha 2, một file có thể bị đóng và slot `struct file` được cấp lại cho file khác, hoặc một tiến trình có thể `fork()`/`exit()`. Vì vậy:

- `fileptr[j]` có thể trỏ tới một slot đã được tái sử dụng; khi đó pid của file mới có thể bị gán nhầm vào entry cũ.
- `p->parent` trên thực tế được bảo vệ bởi `wait_lock`, không phải `p->lock`; đọc dưới `p->lock`

chỉ là best-effort nên `parent_pid` có thể là giá trị cũ.

- `nfile` và `nproc` phản ánh từng bảng tại thời điểm pha tương ứng, không nhất thiết nhất quán tuyệt đối với nhau.

Với mục tiêu quan sát và kiểm thử của lab, các sai lệch này chấp nhận được; chúng được nêu lại ở mục 4.6.

4.0.4. Đếm số lần định thời

`nsched` được thêm vào `struct proc` (`kernel/proc.h:96`), được đặt lại bằng 0 trong `allocproc()` (`kernel/proc.c:127`) và tăng trong `scheduler()` ngay sau khi chuyển trạng thái sang `RUNNING` (`kernel/proc.c:469-470`). Định nghĩa là **số lần scheduler chọn tiến trình**, không phải số lần `yield()` hay số lần ngủ. Việc tăng nằm trong vùng đang giữ `p->lock` của scheduler, nên reader trong `collect()` thấy một giá trị nhất quán. Tiến trình mới có `nsched = 0` cho đến lần chạy đầu tiên; sau đó giá trị luôn dương.

4.0.5. `user/sysinfotest.c` và kết quả

Chương trình kiểm thử gồm **56 check** và **22 lời gọi** `sysinfo()` (21 lần qua `getinfo()` cộng một lần cố tình truyền con trỏ không hợp lệ). Thay vì so sánh với số tuyệt đối, test lấy một snapshot baseline, thực hiện thao tác rồi so sánh **delta** về `nfile`, `ref`, `npid`, `nfd`... nhờ vậy không phụ thuộc vào số file console/README đang mở trên hệ thống.

Nhóm kiểm thử	Số test	Nội dung chính
Baseline và bất biến	12	Có ít nhất một file; có `init`, `sh`, chính test; test `RUNNING`, cha `SLEEPING`; fd 0–2 mở; pid duy nhất, không có `UNUSED`
Console (`FD_DEVICE`)	4	File console được liệt kê, `ref` $\geq$ `3`, đọc/ghi được, nhiều pid tham chiếu
`open` (`FD_INODE`)	6	Tạo đúng một entry, `ref = 1`, `O_RDONLY` được ghi nhận, chỉ một pid
`dup`	5	Không tạo entry mới, `ref = 2`, pid không bị nhân đôi, cả hai fd được liệt kê

Bảng 4.2: Các nhóm kiểm thử trong `user/sysinfotest.c`

Nhóm kiểm thử	Số test	Nội dung chính
`close`	2	`ref` trở về 1; bảng file trở về baseline
`fork`	6	Con được tạo, `ref` tăng, pid con xuất hiện, `parent_pid` đúng, con thừa hưởng fd
`wait`	2	Con rời danh sách; `ref` giảm về 1
`ZOMBIE`	2	Con chưa được `wait` ở trạng thái `ZOMBIE`; sau `wait` biến mất
`SLEEPING`	3	Con bị chặn đọc pipe rỗng ở `SLEEPING`; sau khi được đánh thức thì được reap
`pipe` (`FD_PIPE`)	5	Tạo đúng hai entry, mỗi đầu `ref = 1`, đóng cả hai thì cả hai biến mất
`NOFILE`	5	13 fd còn trống được cấp, 7 lần `open` thất bại, `nfd = 16`, đóng các fd thừa còn 0–2
Cross-check	2	Mọi file có `ref > 0` và danh sách pid; mọi tiến trình có fd đều xuất hiện trong bảng file
`nsched`	1	Tăng sau khi tiến trình ngủ và được chọn lại
`copyout`	1	Con trỏ `MAXVA` bị từ chối, không panic

Bảng 4.3: Các nhóm kiểm thử trong `user/sysinfotest.c` (tiếp theo)

Hai điểm thiết kế đáng chú ý:

- **Console, inode và pipe** được kiểm tra riêng vì chúng đi qua ba nhánh khác nhau của `struct file` (`FD_DEVICE`, `FD_INODE`, `FD_PIPE`). Nhờ vậy test không chỉ kiểm tra số lượng mà còn kiểm tra `type`, `readable`, `writable` và cách `ref` lan truyền.
- **Tiến trình** `SLEEPING` được tạo bằng một con đang chặn trên `read()` của pipe rỗng, không dùng `sleep()`. Lý do là `sleep(n)` bị timer đánh thức mỗi tick, nên trạng thái có thể đổi giữa lúc con ngủ và lúc cha chụp snapshot; chờ pipe thì ổn định hơn.

Kết quả chạy trên QEMU (cấu hình mặc định `-smp 3`):

Mã nguồn 2: `sysinfotest`

```
sysinfotest: sizeof(struct sysinfo) = 32984 bytes
baseline: 1 open files, 3 processes
...
PASS: copyout: invalid user pointer is rejected
ALL TESTS PASSED
```

CHƯƠNG 5

Kết Luận

Phần khảo sát cho thấy xv6 tách biệt ba tầng: file descriptor (chỉ số trong ‘ofile[]‘), open file description (‘struct file‘ trong ‘ftable‘) và inode. Sự tách biệt này lý giải ngữ nghĩa chia sẻ offset của ‘dup‘/‘fork‘ và tính độc lập của nhiều lần ‘open‘ trên cùng một file.

Ở phía tiến trình, ‘pid‘, ‘state‘ và ‘context‘ phối hợp để scheduler chuyển ngữ cảnh an toàn: ‘state‘ quyết định việc được chọn, ‘context‘ đủ để lưu kernel thread, còn ‘trapframe‘ đủ để quay về user space. Đây cũng là cơ sở để cài đặt ‘sysinfo‘: đọc ‘ftable.file[]‘, đọc ‘ofile[]‘ của từng tiến trình và đọc ‘pid‘/‘state‘/‘parent‘ dưới đúng lock.

Phần 4.2 đã hiện thực system call ‘sysinfo‘ với ABI không con trỏ, bộ đệm BSS 32.984 byte và hai pha thu thập dữ liệu không giữ đồng thời ‘ftable.lock‘ và ‘p->lock‘. Chương trình ‘user/sysinfotest.c‘ kiểm tra 56 tình huống thuộc các nhóm console/inode/pipe, ‘open‘/‘dup‘/‘fork‘/‘wait‘, ‘ZOMBIE‘/‘SLEEPING‘, giới hạn ‘NOFILE‘, ‘nsched‘ và kiểm tra con trỏ user không hợp lệ; cả ba lần chạy trên QEMU đều cho 56 PASS, 0 FAIL. Hạn chế chính là snapshot best-effort (có thể lệch nếu bảng thay đổi giữa hai pha) và việc giữ spinlock trong lúc ‘copyout()‘; các hướng cải thiện đã được nêu ở mục 4.6.

TÀI LIỆU THAM KHẢO

[1] MIT PDOS, *xv6-riscv: Xv6 for RISC-V*, GitHub repository. [Online] https://github.com/mit-pdos/xv6-riscv

[2] R. Cox, F. Kaashoek, and R. Morris, xv6: A Simple, Unix-like Teaching Operating System, Revision 4, Aug. 2024. [Online]. https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf

[3] A. Waterman and K. Asanović, Eds., The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, Version 20240411, RISC-V International, 2024. [Online]. https://docs.riscv.org/reference/isa/v20240411/_attachments/riscv-unprivileged.pdf

[4] A. Waterman and K. Asanović, Eds., The RISC-V Instruction Set Manual, Volume II: Priviledged Architecture, Version 20240411, RISC-V International, 2024. [Online]. https://docs.riscv.org/reference/isa/v20240411/_attachments/riscv-privileged.pdf

[5] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.1*, Tech. Rep. UCB/EECS-2016-118, EECS Department, University of California, Berkeley, May 2016. [Online]. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.pdf

DANH MỤC CÁC BẢNG

Bảng 1.1 Phân công và đối chiếu mục nội dung 2

Bảng 2.1 Các trường trong cấu trúc quản lý file 5

Bảng 2.2 Các bước trong luồng `syscall open` 7

Bảng 2.3 So sánh file descriptor trong `open`, `dup`, `fork` 9

Bảng 3.1 Các trạng thái của tiến trình trong `struct proc` 12

Bảng 3.2 So sánh giữa `struct context` và `struct trapframe` 13

Bảng 4.1 Cấu trúc dữ liệu cho hệ thống thống kê tiến trình và file 19

Bảng 4.2 Các nhóm kiểm thử trong `user/sysinfotest.c` 23

Bảng 4.3 Các nhóm kiểm thử trong `user/sysinfotest.c` (tiếp theo) 24

DANH MỤC CÁC HÌNH ẢNH